

Kubernetes Security Detection with Machine Learning

Research and Implementation Document

****Project:**** Predicción de Cadenas de Ataque en Infraestructuras Kubernetes mediante Graph Neural Networks

****Author:**** Obed

****Date:**** April 2026

1. Project Overview

This document outlines the design, scope, and methodology for building a machine learning system capable of detecting security vulnerabilities, predicting attack chains, and identifying escalation paths within Kubernetes infrastructure. The goal is to produce a model that is practical, production-ready, and applicable to real-world cloud environments — not just academic simulations.

The system targets three primary threat categories:

- Configuration misconfigurations (RBAC, network policies, security contexts)
- Lateral movement paths between pods, service accounts, and nodes
- Privilege escalation chains exploiting Kubernetes-specific vulnerabilities

2. Problem Statement

Kubernetes environments are complex and dynamic. Traditional rule-based security scanners catch known misconfigurations but fail to predict how an attacker could chain multiple weaknesses into a full escalation path. The challenge is not just detecting individual issues — it is understanding the structural relationships between components that enable attack chains.

This project addresses that gap using a hybrid machine learning approach that combines statistical classification with graph-based structural learning.

3. Scope and Focus

The system will focus on the following Kubernetes security domains:

- **RBAC Misconfigurations:** Overly permissive roles, wildcard permissions, cluster-admin bindings
- **Pod Security:** Missing security contexts, privileged containers, host path mounts
- **Network Policies:** Missing egress/ingress rules, unrestricted pod-to-pod communication
- **Service Account Abuse:** Default service account usage, token auto-mounting
- **Escalation Paths:** Multi-hop attack chains from a compromised pod to node or cluster admin access
- **Supply Chain Risks:** Unverified images, missing admission controllers

4. Hybrid Model Architecture

The system uses a two-layer hybrid approach, combining the strengths of two fundamentally different model types.

4.1 Layer 1 — Random Forest (Configuration Classifier)

****Purpose:**** Detect individual misconfigurations and score each Kubernetes resource independently.

****Input features:****

- Pod security context flags (privileged, runAsRoot, hostPID, hostNetwork)
- RBAC role permissions (wildcard verbs, resource scope, binding type)
- Network policy presence and coverage per namespace
- Service account token auto-mount settings
- Image pull policy and registry source

****Output:**** Binary classification per resource — Secure / Misconfigured — plus a risk score (0.0 to 1.0).

****Why Random Forest:**** Fast, interpretable, works well on tabular config data, and handles class imbalance effectively. Catches obvious low-hanging-fruit misconfigurations reliably.

4.2 Layer 2 — Graph Neural Network (Attack Chain Predictor)

****Purpose:**** Model the structural relationships between Kubernetes components and predict possible escalation paths.

****Graph representation:****

- Nodes: Pods, Service Accounts, Roles, RoleBindings, Namespaces, Nodes
- Edges: Permission grants, network connectivity, trust relationships
- Node features: Risk scores from Layer 1 plus configuration attributes

****Output:**** Predicted attack paths through the cluster graph — ranked by likelihood and impact.

****Why GNN:**** Attack chains are inherently structural. A compromised pod only becomes dangerous if it has RBAC bindings that allow it to reach sensitive resources. GNNs learn these multi-hop relationship patterns directly from the graph topology.

4.3 Optional Layer 3 — Genetic Algorithm (Ensemble Optimizer)

****Purpose:**** Optimize the weighting between Layer 1 and Layer 2 outputs to maximize detection precision.

****Usage:**** Applied post-training to tune how much weight each model's output carries in the final risk score. Particularly useful when tuning for specific environments such as multi-tenant vs. single-tenant clusters.

5. Training Data Strategy

5.1 Data Requirements

The models require two types of labeled data:

- ****Tabular config data**** for the Random Forest: individual Kubernetes resource manifests labeled as secure or misconfigured.
- ****Graph snapshots**** for the GNN: full cluster state graphs with labeled escalation paths.

5.2 Recommended Approach

Rather than building 500 independent cluster configurations, the recommended strategy is:

1. Design 20–30 realistic base cluster templates, each representing a different production-like scenario (multi-tenant, microservices, CI/CD pipelines, etc.)
2. For each template, enumerate all possible escalation paths — both existing and absent
3. Generate variations of each template by tweaking RBAC rules, network policies, and service account bindings
4. Label each variation with its attack paths, producing 400–500 total labeled examples from a manageable set of base configurations

This approach ensures diversity while keeping data generation feasible.

5.3 Target Volumes

- Random Forest training set: 300–500 labeled resource configs
- GNN training set: 200–300 labeled graph snapshots with attack path annotations
- Validation set: 20% held out from each

5.4 Data Sources

- Synthetic generation from base templates (primary source)
- Real staging environment instrumentation (secondary, for fine-tuning)
- Public references: CIS Kubernetes Benchmark, MITRE ATT&CK for Containers, NSA/CISA Kubernetes Hardening Guide

6. Implementation Roadmap

Phase 1 — Foundation (Weeks 1–3)

- Design 20–30 base cluster configuration templates
- Build annotation tooling to label escalation paths
- Generate initial dataset of 300+ examples

Phase 2 — Random Forest Baseline (Weeks 4–5)

- Train and evaluate the Random Forest classifier
- Target: F1 score above 0.85 on misconfiguration detection
- Analyze failure cases to guide additional data collection

Phase 3 — GNN Development (Weeks 6–9)

- Build graph construction pipeline from Kubernetes manifests
- Train GNN using PyTorch Geometric
- Integrate Random Forest scores as GNN node features
- Target: Precision@5 above 0.70 on attack chain prediction

Phase 4 — Integration and Validation (Weeks 10–12)

- Combine both models into unified detection pipeline
- Validate against real staging environment
- Optionally apply genetic algorithm for ensemble weight optimization
- Document findings and write up results

7. Success Metrics

- **Random Forest:** F1 score above 0.85 on config classification
- **GNN:** Precision@5 above 0.70 on attack chain prediction
- **System:** False positive rate below 10% on clean cluster configurations
- **Practical:** All detections explainable and actionable by an SRE team

8. References

- MITRE ATT&CK for Containers: <https://attack.mitre.org/matrices/enterprise/containers/>
- CIS Kubernetes Benchmark: <https://www.cisecurity.org/benchmark/kubernetes>
- NSA/CISA Kubernetes Hardening Guide
- PyTorch Geometric Documentation: <https://pytorch-geometric.readthedocs.io>
- Kubernetes RBAC Documentation: <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>